

Sender Integration Specifications

Complete Technical Package for API-to-API Settlement

PRODUCTION URL

alshumookh-group-1.onrender.com

INGEST ENDPOINT

POST /api/v1/payloads/ingest

SCHEMA ENDPOINT

GET /api/v1/payloads/schema

DOCUMENT DATE

May 11, 2026

CONFIDENTIAL — FOR AUTHORIZED SENDER USE ONLY

01 Quick Reference

The following table contains everything required to begin integration immediately.

Parameter	Value
Production Base URL	https://alshumookh-group-1.onrender.com
Schema Endpoint	GET /api/v1/payloads/schema
Ingest Endpoint	POST /api/v1/payloads/ingest
Health Check	GET /health
OAuth2 Token	POST /api/v1/oauth/token
Receiver Wallet (ETH)	0xBD682cfD8382a90adfDd6745780D3D7959c4d939
Primary Asset	USDC (ERC-20)
Supported Networks	Ethereum Mainnet, Base Mainnet
USDC Contract (ETH)	0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48
USDC Contract (Base)	0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913
Authentication	X-API-Key header (minimum)
Idempotency Header	Idempotency-Key (required, unique per request)

■ *Note: The domain alshumookh.finance is temporarily under registration review. Use the production URL above until further notice.*

02 Authentication & Security Headers

All requests must be authenticated. The minimum security level is API Key. Elevated security modes (HMAC, OAuth2, JWS, mTLS) can be enabled per counterparty.

Header	Value	Required?	Purpose
Content-Type	application/json	Required	All requests
X-API-Key	<sender_api_key>	Required*	Dedicated key issued by ALSHUMOOKH
Idempotency-Key	unique-string	Required	Prevent duplicate processing
X-Timestamp	ISO-8601 datetime	Recommended	Replay protection
X-Signature	sha256=<hmac_hex>	If HMAC enabled	HMAC-SHA256 of body
Authorization	Bearer <token>	If OAuth2 enabled	Client credentials token
X-JWS-Signature	<jws_compact>	If JWS enabled	Asymmetric signature
X-Client-Cert-Fingerprint	<fingerprint>	If mTLS enabled	Certificate fingerprint

HMAC Signature Construction:

```
base_string = X-Timestamp + '.' + exact_raw_json_body
signature    = HMAC-SHA256(hmac_secret, base_string)
header       = X-Signature: sha256=<hex_digest>
```

03 Full JSON Payload Format

The receiver accepts custom fields. Use the structure below for automatic parsing, blockchain verification, and reconciliation.

```
{
  "transaction_reference": "W2W-2026-000001",
  "transaction_id": "optional-external-id",
  "tx_hash": "0xBLOCKCHAIN_TX_HASH",
  "sender_wallet": "0xSenderWalletAddress",
  "receiver_wallet": "0xBD682cfD8382a90adfDd6745780D3D7959c4d939",
  "amount": "100.00",
  "asset": "USDC",
  "network": "ethereum",
  "token_contract": "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48",
  "timestamp": "2026-05-11T10:00:00Z",
  "sender": {
    "name": "Sender Legal Entity Name",
    "technician": "Technical Contact Name",
    "email": "technical@sender.com"
  },
  "receiver": {
    "name": "ALSHUMOOKH GLOBAL BANKING FINANCE & CREDIT",
    "wallet": "0xBD682cfD8382a90adfDd6745780D3D7959c4d939"
  },
  "authorization_code": "optional-release-code",
  "settlement_type": "wallet_to_wallet",
  "callback_url": "https://sender.example/settlement/callback",
  "metadata": {
    "agreement_reference": "AGR-2026-001",
    "purpose": "settlement",
    "notes": "Custom fields accepted and stored"
  }
}
```

Minimum Required Fields:

Field	Required?	Description
transaction_reference	Yes	Unique external reference (your internal ID)
sender_wallet	Yes	Wallet address funds are sent FROM
receiver_wallet	Yes	Must be ALSHUMOOKH master wallet
amount	Yes	Exact token amount (decimal string)
asset	Yes	Token symbol — e.g. USDC
network	Yes	ethereum / base
token_contract	Recommended	ERC-20 contract address — required for strong verification
tx_hash	Recommended	Blockchain TX hash — if absent, status = AWAITING_TX_HASH
timestamp	Recommended	ISO-8601 UTC — used for audit and replay protection

04 Accepted Field Aliases

If your system uses different field names, the following aliases are automatically recognised:

Canonical Field	Accepted Aliases
tx_hash	hash, txid, transaction_hash, txhash, blockchain_tx
sender_wallet	from_wallet, from, sender, source_wallet
receiver_wallet	to_wallet, to, destination_wallet, beneficiary_wallet
amount	value, token_amount, transfer_amount, settlement_amount
asset	currency, token, symbol, crypto_currency
network	chain, blockchain, protocol

05 Settlement Status Flow

Status	Meaning	Sender Action
RECEIVED	Payload accepted by API	Store the payload_id returned
PARSED	Fields normalised successfully	No action required
AWAITING_TX_HASH	No tx_hash submitted	Submit tx_hash after broadcasting
ALCHEMY_PENDING	Blockchain verification started	Wait for result
ALCHEMY_VERIFIED	Blockchain verification passed	Proceed to reconciliation
ON_CHAIN_CONFIRMED	Transaction confirmed on-chain	Confirm on business side
RECONCILED	Matched to settlement — complete	Close operation
FAILED	Verification or parsing failed	Correct payload or investigate tx
MANUAL_REVIEW	Operator review required	Provide supporting evidence

06 API Response Formats

Successful Payload (200 OK):

```
{
  "status": "payload_received",
  "payload_id": "uuid-string",
  "transaction_reference": "W2W-2026-000001",
  "parsed": true,
  "verification_status": "AWAITING_TX_HASH"
}
```

Unparsed Payload (stored for manual review):

```
{
  "status": "payload_received_unparsed",
  "payload_id": "uuid-string",
  "parsed": false,
  "message": "Payload stored but requires manual review"
}
```

Error Responses:

HTTP Code	Status	Cause
400	Bad Request	Invalid JSON or missing required fields
401	Unauthorized	Missing or invalid X-API-Key
403	Forbidden	IP not in allowlist or security violation
409	Conflict	Duplicate Idempotency-Key — already processed
429	Too Many Requests	Rate limit exceeded — retry after delay
500	Server Error	Contact ALSHUMOOKH technical team

07 Test cURL Examples

Step 1 — Confirm schema (no auth required):

```
curl -X GET \
  https://alshumookh-group-1.onrender.com/api/v1/payloads/schema
```

Step 2 — Submit test payload (minimum fields):

```
curl -X POST \
  https://alshumookh-group-1.onrender.com/api/v1/payloads/ingest \
  -H "Content-Type: application/json" \
  -H "X-API-Key: <YOUR_SENDER_API_KEY>" \
  -H "Idempotency-Key: test-001" \
  -d '{
    "transaction_reference": "W2W-TEST-001",
    "sender_wallet": "0xYourWalletAddress",
    "receiver_wallet": "0xBD682cfD8382a90adfDd6745780D3D7959c4d939",
    "amount": "1.00",
    "asset": "USDC",
    "network": "ethereum",
    "token_contract": "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48"
  }'
```

Step 3 — Submit with tx_hash (after broadcasting):

```
curl -X POST \
  https://alshumookh-group-1.onrender.com/api/v1/payloads/ingest \
  -H "Content-Type: application/json" \
  -H "X-API-Key: <YOUR_SENDER_API_KEY>" \
  -H "Idempotency-Key: test-002" \
  -d '{
    "transaction_reference": "W2W-TEST-002",
    "tx_hash": "0xREAL_BLOCKCHAIN_HASH",
    "sender_wallet": "0xYourWalletAddress",
    "receiver_wallet": "0xBD682cfD8382a90adfDd6745780D3D7959c4d939",
    "amount": "1.00",
    "asset": "USDC",
    "network": "ethereum",
    "token_contract": "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48"
  }'
```

08 Blockchain Verification Scope

ALSHUMOOKH performs internal Alchemy/RPC verification. No Alchemy keys or RPC URLs are shared externally.

Verification Check	Description
TX Hash exists	Confirmed on Ethereum or Base RPC
TX Receipt	Must be successful (status = 1)
ERC-20 Transfer event	Parsed from transaction logs
Receiver wallet	Must match ALSHUMOOKH master wallet
Sender wallet	Matched against declared sender_wallet
Token contract	Matched against declared token_contract
Amount	Matched against declared amount
Block confirmations	Stored — number of confirmations recorded
Explorer URL	Etherscan / Basescan link stored

SECURITY: Do not share Alchemy API keys, RPC URLs with embedded keys, webhook signing secrets, admin keys, or database credentials.

09 Sender Pre-Live Checklist

1	Dedicated sender API key received from ALSHUMOOKH	■
2	Idempotency-Key generation implemented (UUID or unique string per request)	■
3	Security mode confirmed: API Key / HMAC / OAuth2 / JWS / mTLS	■
4	Receiver wallet address confirmed in writing	■
5	Network and token contract confirmed in writing	■
6	Schema endpoint called and response verified	■
7	Small test payload submitted WITHOUT tx_hash — status AWAITING_TX_HASH received	■
8	Small test payload submitted WITH tx_hash — status ALCHEMY_VERIFIED received	■
9	Admin dashboard confirms payload and verification result	■
10	No high-value transfer before small technical test passes completely	■
11	Callback URL tested (if used)	■
12	HMAC/signature verified end-to-end (if HMAC enabled)	■

10 Ready-to-Send Integration Message

Copy and send the following message to your technical counterparty:

Subject: ALSHUMOOKH Settlement Receiver — Technical Integration Package

Dear Technical Team,

Please find attached the complete ALSHUMOOKH Sender Integration Specifications.

Production System:

Base URL: <https://alshumookh-group-1.onrender.com>

Schema: GET /api/v1/payloads/schema

Ingest: POST /api/v1/payloads/ingest

Receiver Wallet (Ethereum/Base):

0xBD682cfD8382a90adfDd6745780D3D7959c4d939

Please review the attached package for full JSON format, authentication headers, status flow, and step-by-step integration process.

Begin with a small technical test payload (minimum 1 USDC) before any high-value transfer. ALSHUMOOKH performs Alchemy/RPC blockchain verification internally.

Kind regards,

ALSHUMOOKH Technical Team

ceo@alshumookhgroup.ae

This document is issued by ALSHUMOOKH GLOBAL BANKING FINANCE & CREDIT. It is confidential and intended only for the authorised sender. Do not distribute without permission. For support: ceo@alshumookhgroup.ae | +971 50-388-8840